HDOC Day-47

Write a C program that reads the size and elements of a sorted variable-length integer array, arr[], in ascending order. Also, read an integer target. Pass the array to a function named ceil_index() using its address. Use pointers to pass the array between the calling function and ceil_index(). The function ceil_index() must:

- Find the index of the smallest element in arr[] that is ≥ target.
- Treat this element as the ceil of target.
- If multiple occurrences of the ceil element exist, return the index of its first occurrence.
- Display the ceil element and its index.
- Display -1 if no element is ≥ target.

Example 1:

Array: 5 7 7 8 8 10
target: 6 (Element 7 is the smallest/ceil element in the given array that is ≥ 6. Element 7 is available in index 1 and 2. The index of 1st occurrence should be returned)

Output:

Ceil of 6: 7 at index 1

Example 2:

Array: 5 7 7 8 9 10
target: 8 (Element 8 is the smallest/ceil element in the given array that is ≥ 8. Element 8 is available in index 3. Return index 3)

Output:

Ceil of 8: 8 at index 3

Example 3:

Array: 5 7 7 8 8 10
target: 11 (There is no element in the given array that is ≥ 11. Return -1)

Output:

-1

1. Algorithm

1. Start.
2. Read the size n of the array.
3. Read n elements in ascending order.
4. Read the integer target.
5. Pass the address of the array and its size to the function ceil_index().
6. Inside the function, traverse the array from index 0 to n-1.
7. If arr[i] >= target, return the index i and the element arr[i]. Since we traverse from left to right, the first occurrence is returned.
8. If no element is greater than or equal to the target, return -1.
9. Display the result.
10. Stop.

C program

```
#include <stdio.h>

int ceil_index(int *arr, int n, int target)

{

   int i;


   for (i = 0; i < n; i++)

   {

      if (*(arr + i) >= target)

      {

         return i;

      }

   }
```

```
    return -1;
}

int main()
{
    int n, target, i, index;

    printf("Enter size of array: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements in ascending order: ");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter target: ");
    scanf("%d", &target);

    index = ceil_index(arr, n, target);

    if (index == -1)
    {
        printf("-1\n");
    }
    else
```

```
    {
        printf("Ceil of %d: %d at index %d\n",
            target, arr[index], index);
    }

    return 0;
}
```

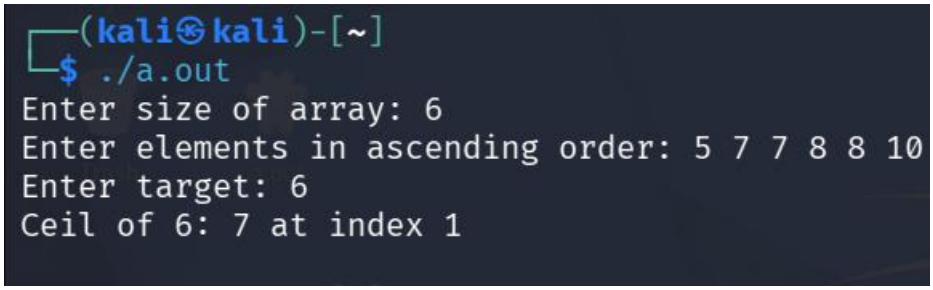
Test Case Table:

Test Case	Array	Target	Expected Output
1	5 7 7 8 8 10	6	Ceil of 6: 7 at index 1
2	5 7 7 8 9 10	8	Ceil of 8: 8 at index 3
3	5 7 7 8 8 10	11	-1
4	2 4 6 8	4	Ceil of 4: 4 at index 1
5	1 3 5 7	0	Ceil of 0: 1 at index 0

Compilation and Execution:

The following program has been compiled and executed successfully.

Test Case 1:



Test Case 2:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size of array: 6
Enter elements in ascending order: 5 7 7 8 9 10
Enter target: 8
Ceil of 8: 8 at index 3
```

Test Case 3:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size of array: 6
Enter elements in ascending order: 5 7 7 8 8 10
Enter target: 11
-1
```

Test Case 4:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size of array: 4
Enter elements in ascending order: 2 4 6 8
Enter target: 4
Ceil of 4: 4 at index 1
```

Test Case 5:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size of array: 4
Enter elements in ascending order: 1 3 5 7
Enter target: 0
Ceil of 0: 1 at index 0
```